

check_fail_open_unanalysed_lang s

check_fail_open_unanalysed_lang s.sh

Revision: 1 **Last modified:** 2026-09-26T14:40:00Z

Overview

scripts/pre_build/check_fail_open_unanalysed_langs.sh is an **advisory** fail-open scan for Go, Rust, Ruby and C (BOB-191). Pre-build invariant 39 runs it over the same first-party roots it passes to CM-DANGEROUS-COMBINATION-FAIL-CLOSED and prints the result as INFO or WARN. It never fails the build.

Why this exists

The constitution scanner enumerates .go, .rs, .rb and .c files but has no analyser for them, so it reports them UNANALYSED. qBitTorrent-go, which holds the encrypted tracker credentials, was 100% unscanned. This script fills the gap until the arms land in the constitution scanner itself (see docs/qa/BOB-191/upstream_proposal_20260926.md).

What it reports

Only shapes that are fail-open by construction, so a validator that returns false on an error (fail-closed) is never reported:

Rule	Language	Shape
	Go (go/ast)	

Rule	Language	Shape
GO-EMPTY-ERR-BRANCH	Go (go/ast)	if err != nil {} — error checked, then dropped
GO-SWALLOWED-PANIC		defer func() { recover() }() — every panic swallowed
RS-EMPTY-ERR-ARM	Rust	Err(..) => {}
RS-EMPTY-IF-LET-ERR	Rust	if let Err(..) = x {}
RB-EMPTY-RESCUE	Ruby	rescue [=> e] followed directly by end
RB-RESCUE-NIL	Ruby	expr rescue nil
C-EMPTY-ERR-BRANCH	C	if (rc != 0) {} on an error-named status variable

Comments and string literals are blanked before matching.

Not covered: a Go error discarded through the blank identifier (`x, _ := f()`), which needs type information (a type-aware tool such as `errcheck`).

Control needles

Before scanning, every analyser runs on a built-in golden-bad needle (must be seen) and a golden-good snippet (must stay silent). If either fails, the script exits 2 rather than reporting zero hits from a blind analyser. A missing `go` or `python3` marks that language UNANALYSED in the summary line.

Usage

```
bash scripts/pre_build/check_fail_open_unanalysed_langs.sh
    qBitTorrent-go cmd/boba-ctl
python3 scripts/pre_build/failopen_lang_analyzer.py --selfcheck
```

Exit codes

Code	Meaning
0	no hits
1	hits (advisory — callers must not block on this)
2	bad arguments, a missing root, a failed control needle, or an unparseable file

Related files

- `scripts/pre_build/failopen_go/` — the Go analyser (stdlib only, built on each run).
- `scripts/pre_build/failopen_lang_analyzer.py` — the Rust/Ruby/C analyser.
- `tests/pre_build/test_check_fail_open_unanalysed_langs.sh` — 15 arms.

Last verified

2026-09-26 — real roots: OK: 0 fail-open hits across 122 Go + 0 Rust/Ruby/C file(s) (control needles seen); test suite 15 passed, 0 failed.